

Fibo AI Stack 模型推理指南

文档版本：V2.0

更新时间：2025 年 12 月 13 日

适用型号

序列	文档版本	适用型号	更新说明
1	1.0	SC171 开发套件 V3	使用 2.24 版本工具链
2	2.0	SC171 开发套件 V3	使用 2.29 版本工具链

目录

1 引言	1
2 所需软硬件环境	2
2.1 硬件环境	2
2.2 软件环境	2
3 详细步骤	2
3.1 准备	2
3.2 Python sample 代码	3
3.3 模型的运行推理	4
4 Q&A	5
4.1 无法找到文件	5
4.2 环境配置失败	5
4.3 推理结果不准确	5

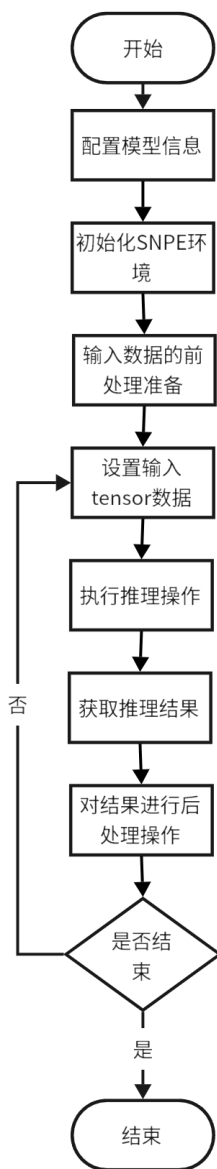
1 引言

本指南详细演示如何在 SC171 开发套件 V3 上使用 Fibocom AI Stack 来快速实现深度学习模型推理

Fibocom AI Stack 提供 AI 工具链、高性能推理引擎、海量模型、支持与服务一体化的端侧 AI 解决方案,帮助用户在 SC171 开发套件中快速实现 AI 能力落地。为适应不同端侧场景的应用, Fibocom AI Stack 提供模型转化工具,支持多种深度学习模型的高效部署。它兼容主流深度学习框架(如 TensorFlow、PyTorch)能够充分发挥硬件性能,提供低延迟、高吞吐量的推理能力。

通过本指南,您不仅能够掌握 Fibo AI Stack 的基础用法,还能根据实际需求灵活更换模型、调整参数,为后续复杂应用的开发奠定坚实基础。若在实践中遇到问题,可随时参考第 4 节的“常见问题解答”或联系技术支持团队。

推理流程图



不同模型的差异操作

在上述的统一流程中，针对不同框架的不同模型文件，也会有一些具体的差异：

- 参数配置：不同模型需要不同的 Model 对象和模型类型、输入输出数据类型等，以告知推理组件该模型的详细信息。
- 数据前处理：不同模型需要不同的输入数据格式，因此前处理操作也会有所不同。
- 数据后处理：不同模型的输出数据格式不同，因此后处理操作也会有所差异。

2 所需软硬件环境

2.1 硬件环境

SC171 开发套件 V3、电脑

2.2 软件环境

Fibo AI Stack SDK、Ubuntu20.04

3 详细步骤

3.1 准备

1. 导入所需的库，在终端输入以下命令：

```
pip3 install <所需安装包名>
```

2. 需要准备的参数：

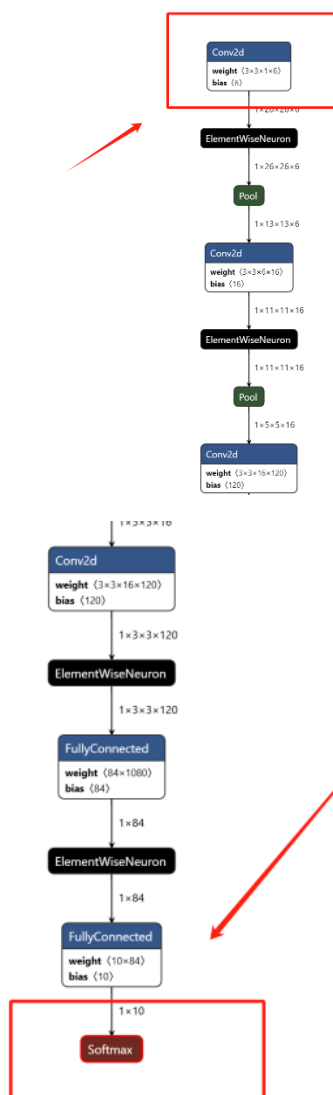
--dlc_path: 模型文件路径（.dlc 文件）

--img_path: 输入文件路径

--模型相关数据:输入张量名称、输出张量名称

3. 将要编译的 Python 文件与 api_infer.py 放在同一路径下

对于模型相关数据不了解的用户，可以通过 [Netron](#) 网站，导入模型文件查看模型相关信息，如图：



NODE PROPERTIES	
type	Conv2d
name	convolution_0
ATTRIBUTES	
bias_op_name	
dilation	tensor: uint32[2]
group	1
packageName	qti.aisw
pad.amount	tensor: uint32[2,2]
padding_size_st...	5
stride	tensor: uint32[2]
INPUTS	
input	name: -serving default conv2d input:0
weight	name: sequential/conv2d/Conv2D
bias	name: sequential/conv2d/BiasAdd/ReadVariableOp
OUTPUTS	
output	name: elementwise_sum_0_0

OUTPUTS	
output	name: StatefulPartitionedCall:0

3.2 Python sample 代码

1. 调用库，定义文件路径
2. 更换推理设备（如 CPU 或 GPU）。
3. 根据使用的模型需求来更改对输入内容的前处理和后处理的代码部分。
4. 修改输入张量名，注意：模型输出内容是键名为模型输出张量名的字典格式的数据。通过模型输出张量名，将数据提取后多为列表格式，要根据模型信息来将列表重塑为数组格式。

```
from api infer import * # Fibo AI推理SDK接口

# ===== 配置区 (用户需修改部分) =====
model_path = "<模型文件路径>"
input_path = "<测试数据输入路径>"
save_path = "<结果保存路径>"
INPUT_NAME = "<模型输入节点名称>"
OUTPUT_NAME = "<模型输出节点名称>"

# ===== 推理引擎配置字典 =====
inference_config = {
    'model': model_path,
    'platform': "qualcomm",
    'framework': "snpe",
    'runtime': "GPU",
    'log_level': "INFO",
    'profile_level': 0,
}

# 创建推理会话, 初始化推理引擎
inference_session = InferenceSession(**inference_config)
assert inference_session.Initialize() == 0, "推理引擎初始化失败"

# 执行预处理
input_data = preprocess(input_path)

# 准备输入数据
input_feed = {INPUT_NAME: input_data}
# 输出层名称列表 (空列表表示获取所有输出)
output_names = [OUTPUT_NAME] if OUTPUT_NAME else []
# 执行推理, 返回包含输出结果的字典
outputs = inference_session.Execute(output_names, input_feed)

# 数据后处理
result = postprocess(outputs, save_path)

# ===== 释放资源 =====
assert inference_session.Release() == 0, "资源释放失败"
```

文件配置区域

推理设备

数据前处理

推理部分

数据后处理

3.3 模型的运行推理

调用 sample 执行文件进行推理, 在开发板终端输入以下命令:

```
python3 <Python sample 文件地址>
```

4 Q&A

4.1 无法找到文件

可能原因:

1. 模型、标签等文件路径配置错误。
2. 模型文件未正确下载或解压。
3. 无权限访问文件。

解决办法:

1. 修改模型的存放路径。
2. 确保模型文件已正确下载并解压到指定路径。
3. 确保运行的 Python 文件路径正确。
4. 检查文件权限，确保当前用户有权限访问模型文件。

4.2 环境配置失败

可能原因:

1. 环境未正确设置。
2. 缺少必要的 Python 包。

解决办法:

1. 检查环境变量是否已正确设置，确保路径中没有错误。
2. 安装必要的 Python 包。

4.3 推理结果不准确

可能原因:

1. 模型输入数据格式不符合模型要求。
2. 模型未正确加载或配置。
3. 模型输出的数据未正确处理。

解决办法:

1. 确保输入数据格式正确使用，可以输出输入数据来检查格式是否符合模型要求：`print(input_feed)`
2. 检查模型，输入输出 Tensor 名称等信息正确。
3. 重新加载模型并运行推理，确保模型配置无误。
4. 检查输入的预处理步骤与后处理步骤，确保数据格式符合模型要求。